

Tasca S3.01. Introducció a BigQuery i SQL al Núvol

Nivell 1: Entorn i Ingesta Híbrida (Code-First)

Escenari:

Acabeu d'arribar a l'empresa. L'equip d'Enginyeria de Dades ha deixat els fitxers històrics al Data Lake corporatiu.

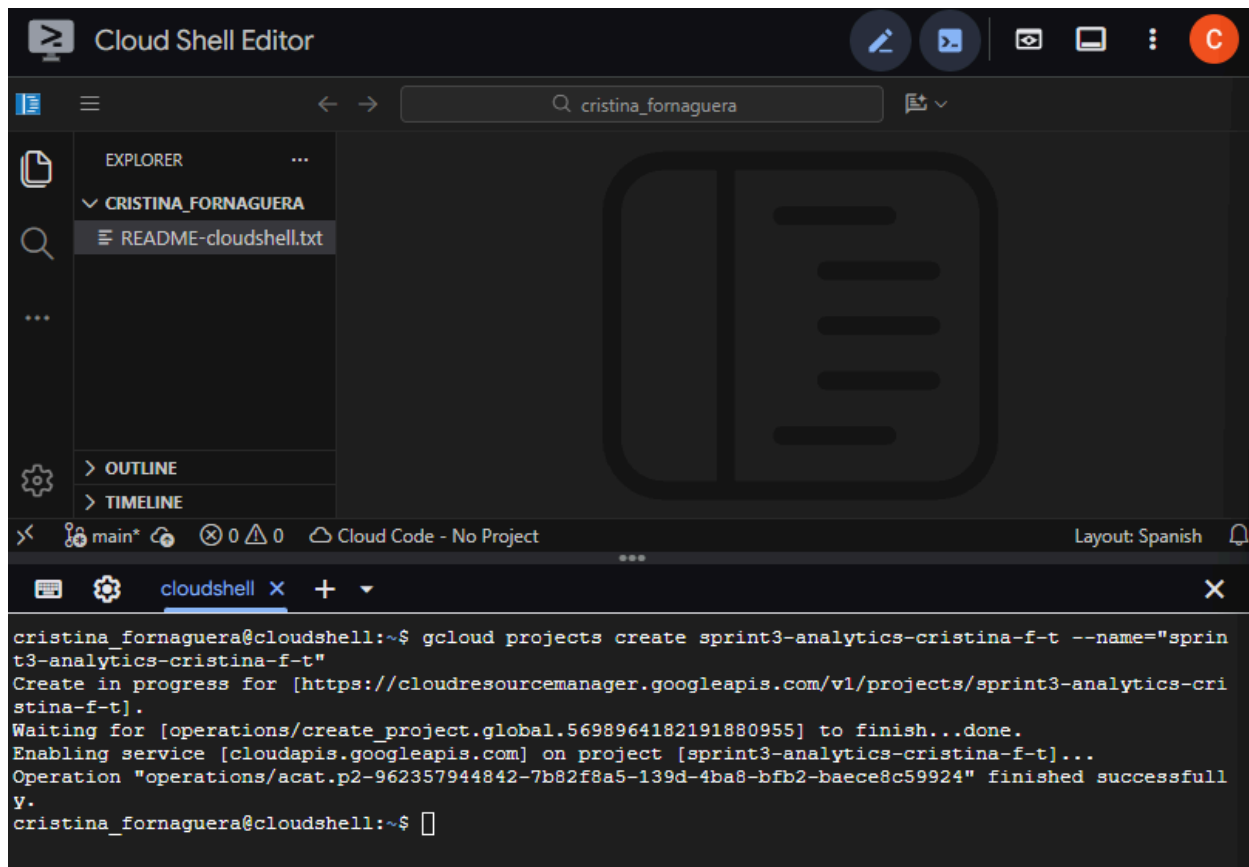
La teva primera missió és dissenyar i crear l'entorn de treball. El CTO exigeix professionalitat: no bolcarem dades sense ordre, sinó que implementarem una Arquitectura Medallion.

Exercici 1: Arquitectura de Dades (Lògica vs. Física)

👉 Crea un Nou Projecte a Google Cloud anomenat: `sprint3-analytics-[el-teu-nom]`.
Ho pots crear per la UI, però si ho fas per Consola (gcloud CLI) té un plus.

- He credo el proyecto 'sprint3-analytics-cristina-f-t'
- Usando el código siguiente en la consola:

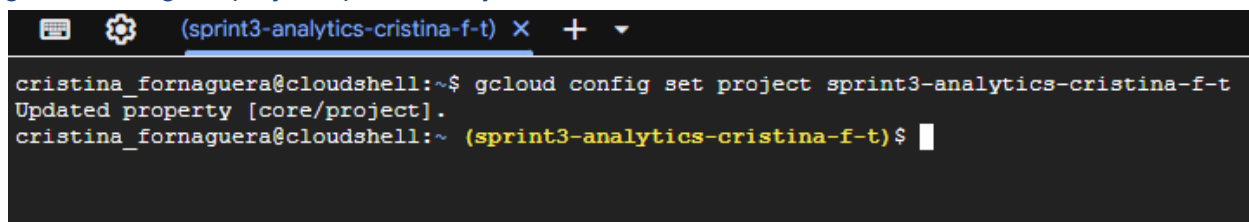
```
gcloud projects create sprint3-analytics-cristina-f-t --name="sprint3-analytics-cristina-f-t"
```



```
cristina_fornaguera@cloudshell:~$ gcloud projects create sprint3-analytics-cristina-f-t --name="sprint3-analytics-cristina-f-t"
Create in progress for [https://cloudresourcemanager.googleapis.com/v1/projects/sprint3-analytics-cristina-f-t].
Waiting for [operations/create_project.global.5698964182191880955] to finish...done.
Enabling service [cloudapis.googleapis.com] on project [sprint3-analytics-cristina-f-t]...
Operation "operations/acat.p2-962357944842-7b82f8a5-139d-4ba8-bfb2-baece8c59924" finished successfully.
cristina_fornaguera@cloudshell:~$
```

- He vinculado la sesión al proyecto a través de la consola:

gcloud config set project sprint3-analytics-cristina-f-t

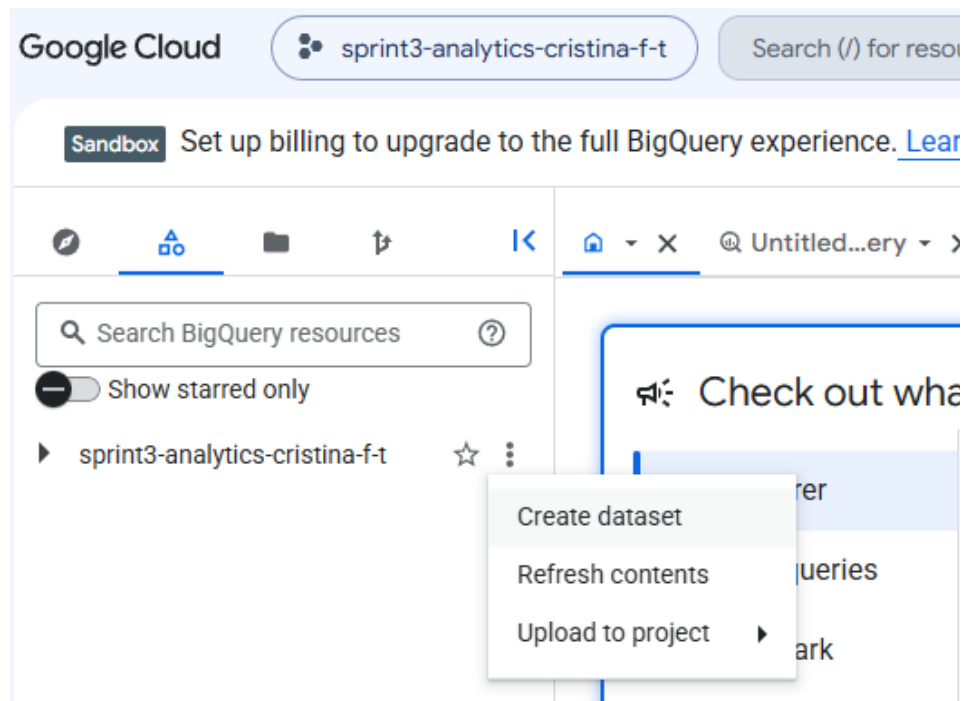


```
(sprint3-analytics-cristina-f-t) $ gcloud config set project sprint3-analytics-cristina-f-t
Updated property [core/project].
(sprint3-analytics-cristina-f-t) $
```

Per demostrar el teu domini total de la plataforma, crearàs cadascun dels datasets utilitzant un mètode diferent:

- » Dataset Físic sprint3_bronze → Capa Lògica Bronze (Dades Crudes)
 - Funció: Dades crues tal com arriben de la font.
 - Mètode: Utilitza la Interfície Gràfica (UI) de BigQuery (fent clics).

- Creando el dataset Bronze usando la UI:



Create dataset

Project ID *
sprint3-analytics-cristina-f-t [Change](#)

Dataset ID *
sprint3_bronze

Letters, numbers, and underscores allowed

Data location
EU (multiple regions in European Union) ▼ ⓘ

External Dataset

The selected region supports the following external dataset types: Cloud Spanner

☐ Link to an external dataset ⓘ

Tags

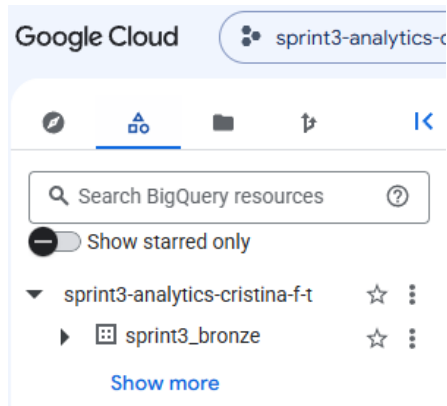


Advanced options



Create dataset

Cancel

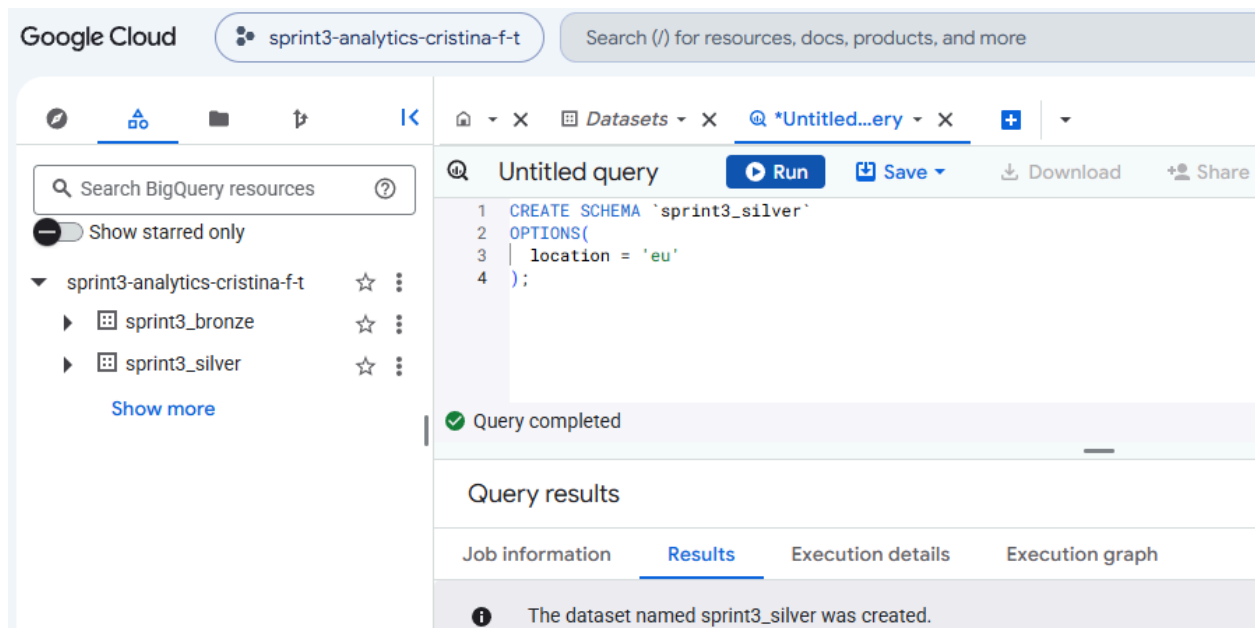


- » Dataset Físic sprint3_silver → Capa Lògica Silver (Dades Netes)
 - Funció: Dades netes, tipades i deduplicades.
 - Mètode: Utilitza codi SQL (CREATE SCHEMA).

- Uso el código siguiente para crear el dataset Silver:

```
CREATE SCHEMA `sprint3_silver`  
OPTIONS(  
  location = 'eu'  
);
```

- Compruebo que se ha creado correctamente:



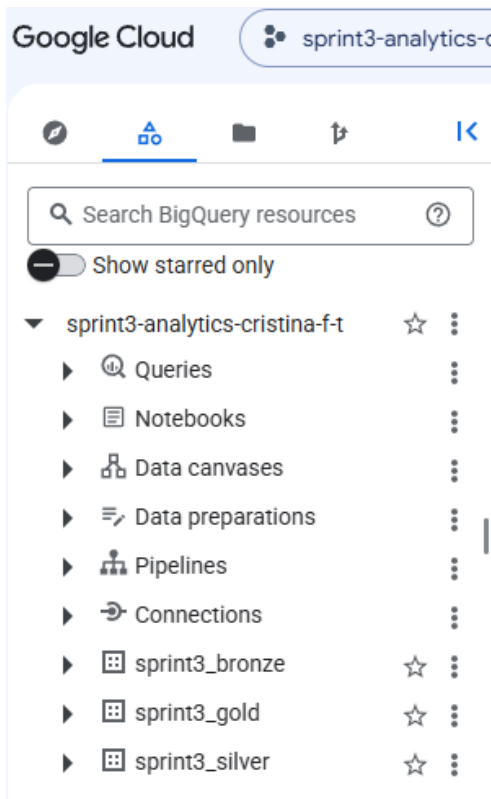
- » Dataset Físic sprint3_gold → Capa Lògica Gold (Dades de Negoci)
 - Funció: Dades agregades llestes per a informes i panells de control (dashboards).
 - Mètode: Utilitza Cloud Shell (Línia d'ordres bq).

- Creo el dataset Gold a través de la shell con el siguiente código:

`bq mk --location=EU sprint3_gold`

```
(sprint3-analytics-cristina-f-t) x + v
cristina_fornaguera@cloudshell:~ (sprint3-analytics-cristina-f-t)$ bq mk --location=EU sprint3_gold
Dataset 'sprint3-analytics-cristina-f-t:sprint3_gold' successfully created.
cristina_fornaguera@cloudshell:~ (sprint3-analytics-cristina-f-t)$
```

- Compruebo en el explorador:



Tingueu en compte la ubicació de les dades originals per decidir la ubicació física dels tres Datasets.

📌 Nota: Encara que siguin capes lògiques diferents, físicament han de viure a la mateixa regió per poder "parlar" entre elles sense costos de transferència.

Exercici 2: Ingesta en Capa Bronze (Connexió DDL)

Un analista professional automatitza les tasques. Ara que tens la teva arquitectura a punt, poblarem la primera capa (Bronze) sense moure les dades originals, creant Taules Externes que apuntin al Data Lake.

🌱 Tasca:

Escriu i executa les sentències CREATE EXTERNAL TABLE per connectar els següents fitxers al dataset sprint3_bronze. Para molta atenció a les "Notes Tècniques", ja que no tots els arxius tenen el mateix format.

Taula	Domini	Ruta al Núvol (URI)	Notes Tècniques
transactions_raw	ERP	gs://bootcamp-data-analytics-public/ERP/transactions.csv	Delimitador: ; (Punt i coma)
companies_raw	ERP	gs://bootcamp-data-analytics-public/ERP/companies.csv	Capçalera: Ignora la 1a fila. Defineix l'esquema manualment perquè tot és text.
american_users_raw	CRM	gs://bootcamp-data-analytics-public/CRM/american_users.csv	Estàndard.
european_users_raw	CRM	gs://bootcamp-data-analytics-public/CRM/european_users.csv	Estàndard.
credit_cards_raw	CRM	gs://bootcamp-data-analytics-public/CRM/credit_cards.csv	Estàndard.

👉 Entrega:

El codi SQL utilitzat per a transactions_raw i una captura de pantalla amb les 5 taules creades.

- Código para crear la tabla 'transactions_raw':

```
CREATE OR REPLACE EXTERNAL TABLE `sprint3_bronze.transactions_raw`  
OPTIONS (  
  format = 'CSV',  
  uris = ['gs://bootcamp-data-analytics-public/ERP/transactions.csv'],  
  field_delimiter = ';' )  
;
```

The screenshot displays the Google Cloud BigQuery interface. On the left, the project 'sprint3-analytics-cristina-f-t' is selected, showing a list of resources including 'sprint3_bronze' and its sub-table 'transactions_raw'. The main panel shows an 'Untitled query' editor with the following SQL code:

```
1 CREATE OR REPLACE EXTERNAL TABLE `sprint3_bronze.transactions_raw`  
2 OPTIONS (  
3   format = 'CSV',  
4   uris = ['gs://bootcamp-data-analytics-public/ERP/transactions.csv'],  
5   field_delimiter = ';' ;  
6 );
```

Below the editor, a status bar indicates 'Query completed'. The 'Query results' section is active, showing a message: 'This statement created a new table named transactions_raw.'

- Como no sabemos la estructura de la segunda tabla 'companies_raw', la cargaremos de forma temporal en una tabla 'test' para ver su estructura y contenido, y después crearemos la tabla 'companies_raw' ya con su propio y adecuado formato:

```
CREATE OR REPLACE EXTERNAL TABLE `sprint3_bronze.test_companies_raw`  
OPTIONS (  
  format = 'CSV',  
  uris = ['gs://bootcamp-data-analytics-public/ERP/companies.csv']  
);
```

S3.01 Cristina Fornaguera Torner

Search BigQuery resources

Show starred only

- sprint3-analytics-cristina-f-t
 - Queries
 - Notebooks
 - Data canvases
 - Data preparations
 - Pipelines
 - Connections
 - sprint3_bronze
 - test_companies_raw
 - transactions_raw

Show more

Untitled query

```
1 CREATE OR REPLACE EXTERNAL TABLE `sprint3_bronze.test_companies_raw`  
2 OPTIONS (  
3   format = 'CSV',  
4   uris = ['gs://bootcamp-data-analytics-public/ERP/companies.csv']  
5 );
```

Query completed

Query results

Job information Results Execution details Execution graph

This statement created a new table named test_companies_raw.

- Hacemos un select para ver los 5 primeros registros:

```
SELECT * FROM `sprint3_bronze.test_companies_raw` LIMIT 5;
```

Untitled query

```
1 CREATE OR REPLACE EXTERNAL TABLE `sprint3_bronze.test_companies_raw`  
2 OPTIONS (  
3   format = 'CSV',  
4   uris = ['gs://bootcamp-data-analytics-public/ERP/companies.csv']  
5 );  
6  
7 SELECT * FROM `sprint3_bronze.test_companies_raw` LIMIT 5;
```

NOTE: Could not compute bytes processed estimate for script.

Query results

Job information Results Visualization JSON Execution details Execution graph

Row	string_field_0	string_field_1	string_field_2	string_field_3	string_field_4	string_field_5
1	company_id	company_name	phone	email	country	website
2	b-2222	Ac Fermentum Incorporated	06 85 56 52 33	donec.porttitor.tellus@yahoo.net	Germany	https://instagram.com/site
3	b-2226	Magna A Neque Industries	04 14 44 64 62	risus.donec.nibh@icloud.org	Australia	https://whatsapp.com/group/9
4	b-2230	Fusce Corp.	08 14 97 58 85	risus@protonmail.edu	United States	https://pinterest.com/sub/cars
5	b-2234	Convallis In Incorporated	06 66 57 29 50	mauris.ut@aol.couk	Germany	https://cnn.com/user/110

```
{  
  "string_field_0": "company_id",  
  "string_field_1": "company_name",  
  "string_field_2": "phone",  
  "string_field_3": "email",  
  "string_field_4": "country",  
  "string_field_5": "website"  
}, {  
  "string_field_0": "b-2222",
```



```
"string_field_1": "Ac Fermentum Incorporated",
"string_field_2": "06 85 56 52 33",
"string_field_3": "donec.porttitor.tellus@yahoo.net",
"string_field_4": "Germany",
"string_field_5": "https://instagram.com/site"
}, {
  "string_field_0": "b-2226",
  "string_field_1": "Magna A Neque Industries",
  "string_field_2": "04 14 44 64 62",
  "string_field_3": "risus.donec.nibh@icloud.org",
  "string_field_4": "Australia",
  "string_field_5": "https://whatsapp.com/group/9"
}, {
  "string_field_0": "b-2230",
  "string_field_1": "Fusce Corp.",
  "string_field_2": "08 14 97 58 85",
  "string_field_3": "risus@protonmail.edu",
  "string_field_4": "United States",
  "string_field_5": "https://pinterest.com/sub/cars"
}, {
  "string_field_0": "b-2234",
  "string_field_1": "Convallis In Incorporated",
  "string_field_2": "06 66 57 29 50",
  "string_field_3": "mauris.ut@aol.couk",
  "string_field_4": "Germany",
  "string_field_5": "https://cnn.com/user/110"
}]
```

- Con estos datos tenemos suficiente para saber los tipos de datos y los nombres de columnas, creamos la tabla 'companies_raw':

```
CREATE OR REPLACE EXTERNAL TABLE `sprint3_bronze.companies_raw`
(
  company_id STRING,
  company_name STRING,
  phone STRING,
  email STRING,
  country STRING,
  website STRING
)
OPTIONS (
  format = 'CSV',
  uris = ['gs://bootcamp-data-analytics-public/ERP/companies.csv'],
  skip_leading_rows = 1
);
```

The screenshot shows the Google Cloud BigQuery interface. On the left is a sidebar with a search bar and a tree view of resources. The main area on the right is titled 'Untitled query' and contains a SQL query. Below the query editor, a status bar indicates 'Query completed'. At the bottom, the 'Query results' section is visible, showing a message: 'This statement created a new table named companies_raw.'

Search BigQuery resources

☐ Show starred only

- sprint3-analytics-cristina-f-t
 - Queries
 - Notebooks
 - Data canvases
 - Data preparations
 - Pipelines
 - Connections
 - sprint3_bronze
 - companies_raw
 - transactions_raw
 - sprint3_gold
 - sprint3_silver

Untitled query [Run] [Save] [Download] [Share]

```

1 CREATE OR REPLACE EXTERNAL TABLE `sprint3_bronze.companies_raw`
2 (
3   company_id STRING,
4   company_name STRING,
5   phone STRING,
6   email STRING,
7   country STRING,
8   website STRING
9 )
10 OPTIONS (
11   format = 'CSV',
12   uris = ['gs://bootcamp-data-analytics-public/ERP/companies.csv'],
13   skip_leading_rows = 1
14 );

```

✓ Query completed

Query results

Job information | **Results** | Execution details | Execution graph

i This statement created a new table named companies_raw.

- Creamos el resto de tablas importando los .csv:

```

CREATE OR REPLACE EXTERNAL TABLE `sprint3_bronze.american_users_raw`
OPTIONS (
  format = 'CSV',
  uris = ['gs://bootcamp-data-analytics-public/CRM/american_users.csv']
);

```

```

CREATE OR REPLACE EXTERNAL TABLE `sprint3_bronze.european_users_raw`
OPTIONS (
  format = 'CSV',
  uris = ['gs://bootcamp-data-analytics-public/CRM/european_users.csv']
);

```

```

CREATE OR REPLACE EXTERNAL TABLE `sprint3_bronze.credit_cards_raw`
OPTIONS (
  format = 'CSV',
  uris = ['gs://bootcamp-data-analytics-public/CRM/credit_cards.csv']
);

```

Search BigQuery resources

Show starred only

sprint3-analytics-cristina-f-t

- Queries
- Notebooks
- Data canvases
- Data preparations
- Pipelines
- Connections
- sprint3_bronze
 - american_users_raw
 - companies_raw
 - credit_cards_raw
 - european_users_raw
 - transactions_raw
- sprint3_gold
- sprint3_silver

Search BigQuery resources

Untitled query

```

1 CREATE OR REPLACE EXTERNAL TABLE `sprint3_bronze.american_users_raw`
2 OPTIONS (
3   format = 'CSV',
4   uris = ['gs://bootcamp-data-analytics-public/CRM/american_users.csv']
5 );
6
7 CREATE OR REPLACE EXTERNAL TABLE `sprint3_bronze.european_users_raw`
8 OPTIONS (
9   format = 'CSV',
10  uris = ['gs://bootcamp-data-analytics-public/CRM/european_users.csv']
11 );
12
13 CREATE OR REPLACE EXTERNAL TABLE `sprint3_bronze.credit_cards_raw`
14 OPTIONS (
15   format = 'CSV',
16   uris = ['gs://bootcamp-data-analytics-public/CRM/credit_cards.csv']
17 );

```

Query completed

All results

Elapsed time	Statements processed	Job status
5 sec	3	SUCCESS

Status	End time	SQL	Sta
✓	7:47 PM	CREATE OR REPLACE EXTERNAL TABLE `sprint3_bronze.american_users_raw`	0

sprint3-analytics-cristina-f-t / Datasets / sprint3_bronze

sprint3_bronze

Create Table Chat Preview Share

Overview Details Insights Preview

Tables Graphs Models Routines

Filter Enter property name or value

Table ID	Type	Create time
american_users_raw	Table	Jun 3, 2026, 7:47...
companies_raw	Table	Jun 3, 2026, 7:45...
credit_cards_raw	Table	Jun 3, 2026, 7:47...
european_users_raw	Table	Jun 3, 2026, 7:47...
transactions_raw	Table	Jun 3, 2026, 7:22...

Exercici 3: Càrrega de Dades Locals (Upload)

Falta el catàleg de Productes. Aquesta informació no és al Data Lake, utilitza el fitxer products.csv de l'sprint 2.

- » Carrega aquest fitxer manualment ("Upload") al dataset sprint3_bronze i crea la taula products_raw.
- » Observa que aquesta serà l'única Taula Nativa de moment.

- Recuperamos el fichero products.csv del sprint 2 y lo usamos para crear la tabla nativa 'products_raw'

Create table

Source

Create table from
Upload

Select file *
products.csv

File format
CSV

Destination

Project *
sprint3-analytics-cristina-f-t

Dataset *
sprint3_bronze

Table *
products_raw

Maximum name size is 1,024 UTF-8 bytes. Unicode letters, marks, numbers, connectors, dashes, and spaces are allowed.

Table type
Native table

Create table

Cancel

sprint3-analytics-cristina-f-t / Datasets / sprint3_bronze

sprint3_bronze

Create Table

Chat

Preview

Share

Copy

Del

Overview

Details

Insights

Preview

Tables

Graphs

Models

Routines

Filter

Enter property name or value

Table ID	Type	Create time	Expira
american_users_raw	Table	Jun 3, 202...	Aug 2,
companies_raw	Table	Jun 3, 202...	Aug 2,
credit_cards_raw	Table	Jun 3, 202...	Aug 2,
european_users_raw	Table	Jun 3, 202...	Aug 2,
products_raw	Table	Jun 3, 202...	Aug 2,
transactions_raw	Table	Jun 3, 202...	Aug 2,

- Vemos que la tabla es nativa porque sí que ocupa espacio de Storage:

sprint3-analytics-cristina-f-t / Datasets / sprint3_bronze / Tables / products_raw

☆ products_raw 🔍 Query 🗨 Chat 📄 Open in ▾ 👤 Share ▾ 📄

Schema Details Preview Table Explorer **Preview** Insights Lineage

Tags

Storage info ⓘ

Number of rows	100
Total logical bytes	8.88 KB
Active logical bytes	8.88 KB
Long term logical bytes	0 B
Current physical bytes	0 B
Total physical bytes	0 B
Active physical bytes	0 B
Long term physical bytes	0 B
Time travel physical bytes	0 B

- En cambio en una tabla externa, en esa misma zona se muestran los datos externos en lugar de en Storage:

sprint3-analytics-cristina-f-t / Datasets / sprint3_bronze / Tables / companies_raw

☆

companies_raw

🔍 Query

💬 Chat

Open in ▾

+ Share ▾

Schema

Details

Insights

Lineage

Data Profile

Data Quality

Table ID

sprint3-analytics-cristina-f-t:sprint3_bronze:companies_raw

Created

Jun 3, 2026, 7:45:34 PM UTC+2

Last modified

Jun 3, 2026, 7:45:34 PM UTC+2

Table expiration

Aug 2, 2026, 7:45:34 PM UTC+2

Data location

EU

Case insensitive

false

Description

Labels

Primary key(s)

Tags

External Data Configuration

Source URI(s)

gs://bootcamp-data-analytics-public/ERP/companies.csv

Source format

CSV

Skip leading rows (CSV)

1

Exercici 4: Arquitectura i Rendiment. Materialització de Dades (Assistit per IA)

Escenari:

Les consultes sobre el Data Lake van lentes. Has de demostrar al teu mànager la diferència de rendiment entre treballar amb fitxers externs i treballar amb taules natives de BigQuery.

a) Materialització de Dades (Assistit per IA)

🛠️ Tasca:

Crea una nova taula anomenada `sprint3_bronze.transactions_raw_native` que sigui una còpia exacta de la teva taula externa `transactions_raw`.

Requisit de "AI-Pair Programming":

No escriguis el codi SQL manualment. Utilitza l'Assistent d'IA (Gemini) integrat a BigQuery (o el xat lateral) per generar la sentència.

👉 El Teu Rol: Ets el responsable de validar que el codi que et dona la IA és correcte abans d'executar-lo.

Prompt Suggestit:

"Write a SQL query to create a new table called transactions_raw_native in the sprint3_bronze dataset. It should contain all data from the transactions_raw table. Please use CREATE OR REPLACE TABLE so I don't get errors if I run it more than once."

👉 Entrega:

Una captura de pantalla on es vegi:

- » El prompt que vas escriure a la IA.
- » El codi SQL que la IA va generar.
- » El resultat de l'execució ("This statement created a new table...").

The screenshot shows the Google Cloud BigQuery interface. On the left, the project 'sprint3-analytics-cristina-f-t' is selected, and the 'sprint3_bronze' dataset is visible in the sidebar. The main panel displays an 'Untitled query' with the following SQL code:

```
1 CREATE OR REPLACE TABLE `sprint3-analytics-cristina-f-t.sprint3_bronze.transactions_raw_native`
2 AS
3 SELECT * FROM `sprint3-analytics-cristina-f-t.sprint3_bronze.transactions_raw`;
4
5
```

The query has been executed successfully, as indicated by the 'Query completed' status. The 'Query results' section shows a message: 'This statement created a new table named transactions_raw_native.' with a 'Go to table' button.

On the right, the 'Cloud Assist' sidebar shows a chat interface. The prompt is: 'Write a SQL query to create a new table called transactions_raw_native in the sprint3_bronze dataset. It should contain all data from the transactions_raw table. Please use CREATE OR REPLACE TABLE so I don't get errors if I run it more than once.' The AI response is: 'I've proposed a query to create the transactions_raw_native table in your sprint3_bronze dataset by selecting all data from transactions_raw.' Below the response, the generated SQL code is shown, matching the code in the main panel.

b) Auditoria de Costos

🌱 Tasca:

Dissenyar una prova per comparar el cost de llegir una sola columna (ex: id) a la taula externa vs. la taula nativa. Utilitza els Bytes processats i els Bytes facturats per fer la comparació.

- » Quant "pesa" la consulta a transactions_raw?
- » Quant "pesa" la mateixa consulta a transactions_raw_native?

👉 Entrega:

Captures del validador mostrant la diferència i una conclusió tècnica.

SELECT id

FROM `sprint3\_bronze.transactions\_raw`;

The image shows two screenshots of the Google BigQuery interface. The top screenshot shows a query titled 'Untitled query' with the following SQL code:

```
1 SELECT id
2 FROM `sprint3_bronze.transactions_raw`;
```

Below the query, a status bar indicates: "This query will process 0 B when run." The bottom screenshot shows another query titled 'Untitled query' with the following SQL code:

```
1 SELECT id
2 FROM `sprint3_bronze.transactions_raw_native`;
```

Below this query, a status bar indicates: "This query will process 3.62 MB when run."

- Vemos que en la tabla externa el coste del select sería de 0B mientras que en la tabla nativa sería de 3.62MB
- Conclusión técnica :
- En el Dry Run de la tabla **nativa** (3.62MB), BQ 'sabe' el coste de la operación porque tiene los datos incorporados en su Storage, está optimizada a nivel columnar y puede calcular exactamente cuántos bytes procesará.
- En cambio, en el Dry Run de la tabla **externa** (0B) BQ no puede prever o calcular el tamaño de un fichero externo a Storage, pero en realidad consume más recursos en la

ejecución. No sabe cuánto costará realmente la consulta hasta que acceda al fichero, lo abra y lo ejecute, y por defecto muestra que la consulta costará 0B aunque no lo haya calculado realmente, es decir, 'aún no lo sabe'.

c) El perill del LIMIT

Molts analistes pensen que posar LIMIT 10 redueix el cost de la consulta.

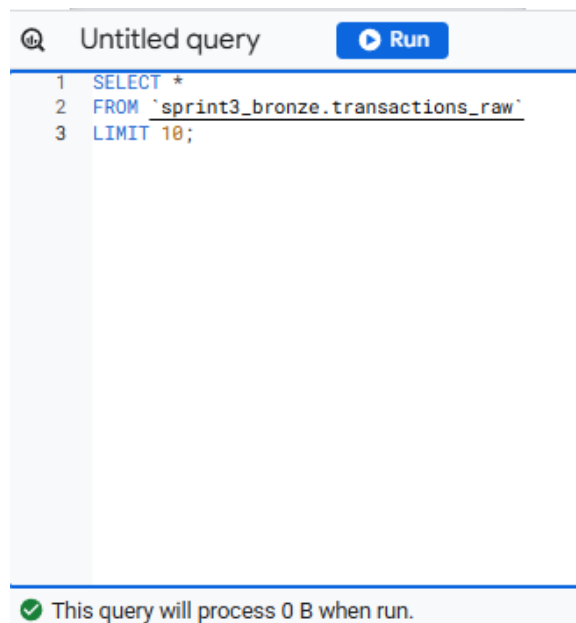
✖ Tasca:

Fes un SELECT * FROM transactions_raw_**native** LIMIT 10 sobre la taula **externa**.

✓ Comprovació: Mira el "Dry Run". Ha baixat el cost respecte a no posar LIMIT? Explica breument per què això és una trampa per a principiants.

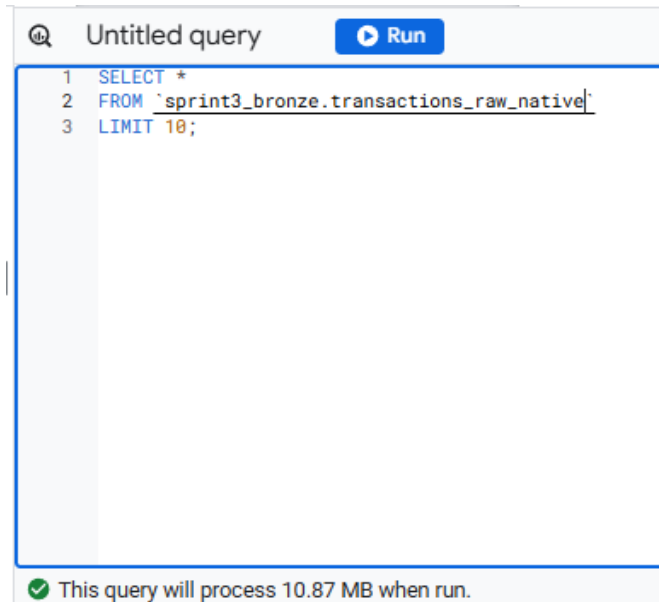
- ** Este enunciado es confuso: indica la tabla **nativa** en el select pero en cambio dice que se ejecute sobre la tabla **externa**. Ante esta incongruencia mostre aremos los resultados del Dry Run de las dos tablas.

```
SELECT *  
FROM `sprint3_bronze.transactions_raw`  
LIMIT 10;
```



- Igual que en el caso anterior, y por el mismo motivo, muestra 0B.
- Hacemos el select en la tabla nativa:

```
SELECT *  
FROM `sprint3_bronze.transactions_raw_native`  
LIMIT 10;
```



The screenshot shows a SQL query editor titled "Untitled query" with a "Run" button. The query is as follows:

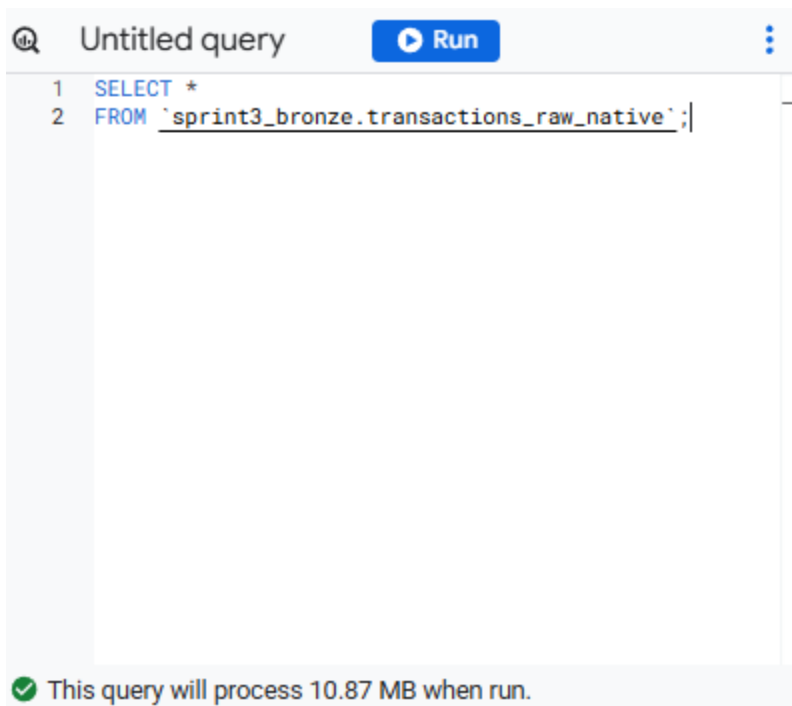
```
1 SELECT *
2 FROM `sprint3_bronze.transactions_raw_native`
3 LIMIT 10;
```

Below the query, a status message indicates: "This query will process 10.87 MB when run."

- En este caso, el Dry Run muestra 10.87MB, más MB que al seleccionar sólo el id, ya que debe procesar todos los datos de la tabla (*) pero mostrará sólo 10 registros. En la consulta del id sólo cargaba los datos de la columna 'id' y por eso el coste de procesamiento era menor.

SELECT *

FROM `sprint3\_bronze.transactions\_raw\_native`;



The screenshot shows a SQL query editor titled "Untitled query" with a "Run" button. The query is as follows:

```
1 SELECT *
2 FROM `sprint3_bronze.transactions_raw_native`;
```

Below the query, a status message indicates: "This query will process 10.87 MB when run."

- Al hacer el mismo select sin el LIMIT, vemos que el coste es exactamente el mismo, dado que debe hacer la misma operación de carga de todos los datos,

independientemente de la cantidad de datos que muestre después que, en este caso, serían todos los registros.

- La 'Trampa' es que el coste es el mismo haciendo la consulta con y sin LIMIT, ya que el motor escanea el 100% de las columnas de la tabla antes de aplicar el filtro de registros a mostrar (LIMIT 10).

Exercici 5: Adaptació de Sintaxi (Reporting)

El teu cap vol saber quins van ser els 5 dies amb més ingressos de l'any 2021.

💪 Repte:

Probablement el camp timestamp és un STRING. Hauràs d'investigar funcions de BigQuery (SUBSTR, CAST, PARSE_TIMESTAMP) per filtrar l'any i agrupar per data correctament

- Primero averiguaremos qué tipo de datos tienen las columnas, haremos la consulta siguiente en la tabla **nativa** para menos coste de procesamiento que haciendo la consulta en la tabla externa:

```
SELECT
  column_name,
  data_type
FROM
  `sprint3_bronze.INFORMATION_SCHEMA.COLUMN_FIELD_PATHS`
WHERE
  table_name = 'transactions_raw_native';
```

Untitled query Run Save Download Share

```

1 SELECT
2   column_name,
3   data_type
4 FROM
5   `sprint3_bronze.INFORMATION_SCHEMA.COLUMN_FIELD_PATHS`
6 WHERE
7   table_name = 'transactions_raw_native';

```

✓ This query will process 10 MB when run.

Using on-demand processing quota

Query results

Job information	Results	Visualization	JSON	Execution details	E:
Row	column_name	data_type			
1	id	STRING			
2	card_id	STRING			
3	business_id	STRING			
4	timestamp	TIMESTAMP			
5	amount	FLOAT64			
6	declined	INT64			
7	product_ids	STRING			
8	user_id	INT64			
9	lat	FLOAT64			
10	longitude	FLOAT64			

- Podemos observar que el campo 'timestamp' es de tipo TIMESTAMP y 'amount' es de tipo FLOAT64. Ya podemos crear la consulta que muestre los 5 días con más ingresos del año 2021. Usaremos la función EXTRACT para seleccionar sólo el año del timestamp, el código es el siguiente:

```

SELECT
  DATE(timestamp) AS data_dia,
  ROUND(SUM(amount),2) AS ingresos_totals
FROM
  `sprint3_bronze.transactions_raw_native`
WHERE
  EXTRACT(YEAR FROM timestamp) = 2021
GROUP BY
  data_dia
ORDER BY

```

ingressos_totals DESC
LIMIT 5;

The screenshot shows a SQL query editor with the following query:

```

1 SELECT
2   DATE(timestamp) AS data_dia,
3   ROUND(SUM(amount),2) AS ingressos_totals
4 FROM
5   `sprint3_bronze.transactions_raw_native`
6 WHERE
7   EXTRACT(YEAR FROM timestamp) = 2021
8 GROUP BY
9   data_dia
10 ORDER BY
11   ingressos_totals DESC
12 LIMIT 5;

```

Below the query, a status bar indicates "Query completed" and "Using on-demand processing quota".

The "Query results" section shows a table with 5 rows and 2 columns: data_dia and ingressos_totals.

Row	data_dia	ingressos_totals
1	2021-10-27	11440.81
2	2021-01-22	11328.84
3	2021-02-11	11182.16
4	2021-04-29	11119.01
5	2021-03-27	10898.51

Exercici 6: Consultes Complexes

Necessitem un informe que creui dades.

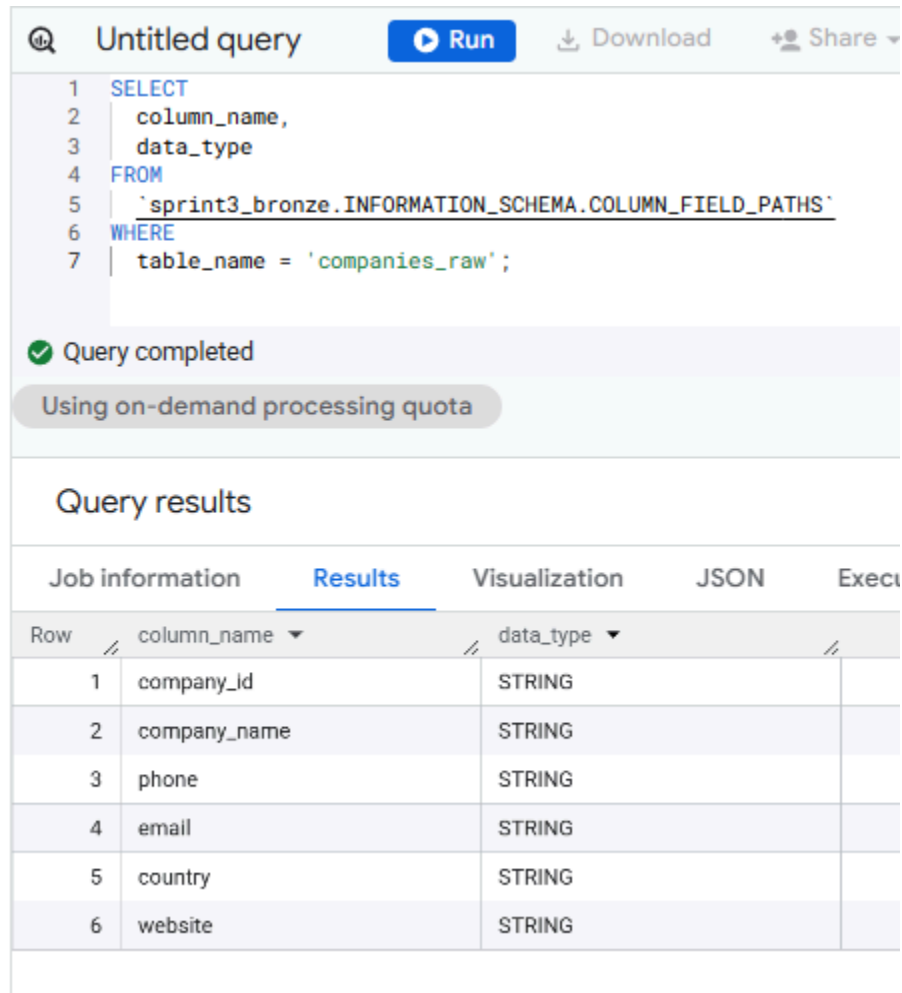
✖ Tasca:

Llista el nom, país i data de les transaccions realitzades per empreses que van fer operacions entre 100 i 200 euros en alguna d'aquestes dates: 29-04-2015, 20-07-2018 o 13-03-2024.

- Para este informe deberemos consultar las tablas siguientes:
- `sprint3_bronze.transactions_raw_native` ← nativa y menos coste de procesamiento
- `sprint3_bronze.companies_raw` ← datos de empresas

- Como hasta ahora no hemos utilizado la tabla 'companies_raw', conviene saber su **esquema** para crear la consulta:

```
SELECT
  column_name,
  data_type
FROM
  `sprint3_bronze.INFORMATION_SCHEMA.COLUMN_FIELD_PATHS`
WHERE
  table_name = 'companies_raw';
```



Untitled query [Run] [Download] [Share]

```
1 SELECT
2   column_name,
3   data_type
4 FROM
5   `sprint3_bronze.INFORMATION_SCHEMA.COLUMN_FIELD_PATHS`
6 WHERE
7   table_name = 'companies_raw';
```

✓ Query completed

Using on-demand processing quota

Query results

Job information	Results	Visualization	JSON	Execu
Row	column_name	data_type		
1	company_id	STRING		
2	company_name	STRING		
3	phone	STRING		
4	email	STRING		
5	country	STRING		
6	website	STRING		

- Ahora que ya tenemos toda la información podemos crear la consulta:

```
SELECT
  c.company_name AS nom_empresa,
  c.country AS pais,
  DATE(t.timestamp) AS data_transaccio,
  ROUND(t.amount,2) AS import
FROM
  `sprint3_bronze.transactions_raw_native` AS t
JOIN
```

```
`sprint3_bronze.companies_raw` AS c
ON t.business_id = c.company_id
WHERE
t.amount BETWEEN 100 AND 200
AND (
DATE(t.timestamp) = '2015-04-29'
OR DATE(t.timestamp) = '2018-07-20'
OR DATE(t.timestamp) = '2024-03-13'
)
ORDER BY
data_transaccio ASC;
```

✔ This query will process 2.29 MB when run.

Untitled query

Run
 Save
 Download
 Share
 Schedule
 Open

```

1 SELECT
2   c.company_name AS nom_empresa,
3   c.country AS pais,
4   DATE(t.timestamp) AS data_transaccio,
5   ROUND(t.amount,2) AS import
6 FROM
7   `sprint3_bronze.transactions_raw_native` AS t
8 JOIN
9   `sprint3_bronze.companies_raw` AS c
10  ON t.business_id = c.company_id
11 WHERE
12   t.amount BETWEEN 100 AND 200
13  AND (
14    DATE(t.timestamp) = '2015-04-29'
15    OR DATE(t.timestamp) = '2018-07-20'
16    OR DATE(t.timestamp) = '2024-03-13'
17  )
18 ORDER BY
19   data_transaccio ASC;
```

✔ This query will process 2.29 MB when run.

Using on-demand processing quota

Query results Chi

Job information	Results	Visualization	JSON	Execution details	Execution graph
Row	nom_empresa	pais	data_transaccio	import	
1	Dictum Eu Corp.	Canada	2015-04-29	177.67	
2	Auctor Mauris Vel LLP	United States	2015-04-29	181.42	
3	Donec Fringilla PC	France	2015-04-29	187.52	
4	Non Magna LLC	United Kingdom	2015-04-29	178.1	
5	Ac Libero Inc.	United Kingdom	2015-04-29	124.04	
6	Amet Lorem LLP	Spain	2015-04-29	145.89	

7	Tempor Diam Institute	Netherlands	2018-07-20	124.04	
8	Tempor Diam Institute	Netherlands	2018-07-20	124.04	
9	Et Magnis Ltd	Belgium	2018-07-20	114.44	
10	Justo Eu Arcu Ltd	Italy	2018-07-20	136.6	
11	Netus Et Malesuada Ltd	Netherlands	2018-07-20	103.73	
12	Aliquam Erat Volutpat LLP	Italy	2018-07-20	127.09	
13	Netus Et Malesuada Ltd	Netherlands	2018-07-20	183.64	
14	Fusce Corp.	United States	2018-07-20	114.77	
15	At Pede Corp.	Italy	2018-07-20	115.16	
16	Magna Incorporated	New Zealand	2018-07-20	144.16	
17	Aliquam Erat Volutpat LLP	Italy	2024-03-13	128.48	
18	Ac Libero Inc.	United Kingdom	2024-03-13	172.88	
19	Justo Eu Arcu Ltd	Italy	2024-03-13	114.09	

Results per page: 50 ▼ 1 – 19 of 19 |<

Nivell 2: Neteja i Transformació (ELT)

Escenari:

Les dades brutes tenen problemes de qualitat (símbols de moneda, usuaris duplicats, dates incorrectes). Actua com a Data Engineer per netejar-ho tot.

Exercici 1: Neteja de Productes (Data Quality)

Crearem la capa neta ("Silver") per als productes. Tens la taula `sprint3_bronze.products_raw` carregada des del CSV. L'equip de Data Governance et demana crear una taula de productes neta a `sprint3_silver.products_clean` que compleixi aquestes regles de qualitat:

- » Estandardització de Noms: La columna original `id` és ambigua; reanomena-la a `product_id`. La columna `product_name` simplifica-la a `name`.
- » Neteja d'IDs: El camp `warehouse_id` té el format antic "WH-4". Elimina el prefix "WH-" i converteix el valor a número enter (INT64).
- » Garantia de Preu: Assegura't que `price` és un número (FLOAT64), sense símbols de moneda.
- » Altres columnes: Conserva el camp `weight` (pes) tal qual.

- Empezamos por averiguar el esquema de la tabla 'products_raw':

```
SELECT
  column_name,
  data_type
FROM
  `sprint3_bronze.INFORMATION_SCHEMA.COLUMN_FIELD_PATHS`
WHERE
  table_name = 'products_raw';
```


Untitled query Run Save Download Share

```
1 SELECT
2   column_name,
3   data_type
4 FROM
5   `sprint3_bronze.INFORMATION_SCHEMA.COLUMN_FIELD_PATHS`
6 WHERE
7   table_name = 'products_raw';
```

✓ Query completed

Using on-demand processing quota

Query results

Job information	Results	Visualization	JSON	Execution details
Row	column_name	data_type		
1	id	INT64		
2	product_name	STRING		
3	price	FLOAT64		
4	colour	STRING		
5	weight	FLOAT64		
6	warehouse_id	STRING		
7	category	STRING		
8	brand	STRING		
9	cost	FLOAT64		
10	launch_date	DATE		

- Una vez conocemos los tipos de datos que contiene, vemos que no es necesario convertir la columna 'price' a FLOAT64 porque ya lo es de origen. El código queda así:

```
CREATE OR REPLACE TABLE `sprint3_silver.products_clean` AS
SELECT
  id AS product_id,
  product_name AS name,
  CAST(REPLACE(warehouse_id, 'WH-', '') AS INT64) AS warehouse_id,
  price,
  weight,
  colour,
  category,
  brand,
```

```
cost,  
launch_date  
FROM  
`sprint3_bronze.products_raw`;
```

The screenshot shows a SQL query editor interface. At the top, there's a toolbar with buttons for 'Run', 'Save', 'Download', and 'Share'. The query text is as follows:

```
1 CREATE OR REPLACE TABLE `sprint3_silver.products_clean` AS  
2 SELECT  
3   id AS product_id,  
4   product_name AS name,  
5   CAST(REPLACE(warehouse_id, 'WH-', '' ) AS INT64) AS warehouse_id,  
6   price,  
7   weight,  
8   colour,  
9   category,  
10  brand,  
11  cost,  
12  launch_date  
13 FROM  
14 `sprint3_bronze.products_raw`;
```

Below the query editor, a status bar indicates 'Query completed' with a green checkmark. A message below that says 'Using on-demand processing quota'. The 'Query results' section is visible, with tabs for 'Job information', 'Results', 'Execution details', and 'Execution graph'. The 'Results' tab is selected, showing a message: 'This statement created a new table named products_clean.'

Exercici 2: Creació de Transaccions Netes (Capa Silver)

Escenari:

La taula `transactions_raw` (Bronze) és insegura: l'import (`amount`) podria tenir lletres, les coordenades podrien fallar i la data és un text difícil de filtrar. Crearem la taula definitiva `sprint3_silver.transactions_clean` que garanteixi el següent.

- » Estandardització de Noms: La columna original `id` és ambigua; reanomena-la a `transaction_id`.
- » Robustesa en Imports: Utilitza `SAFE_CAST` al camp `amount`. Si falla, substitueix-lo per 0 (usant `IFNULL`).

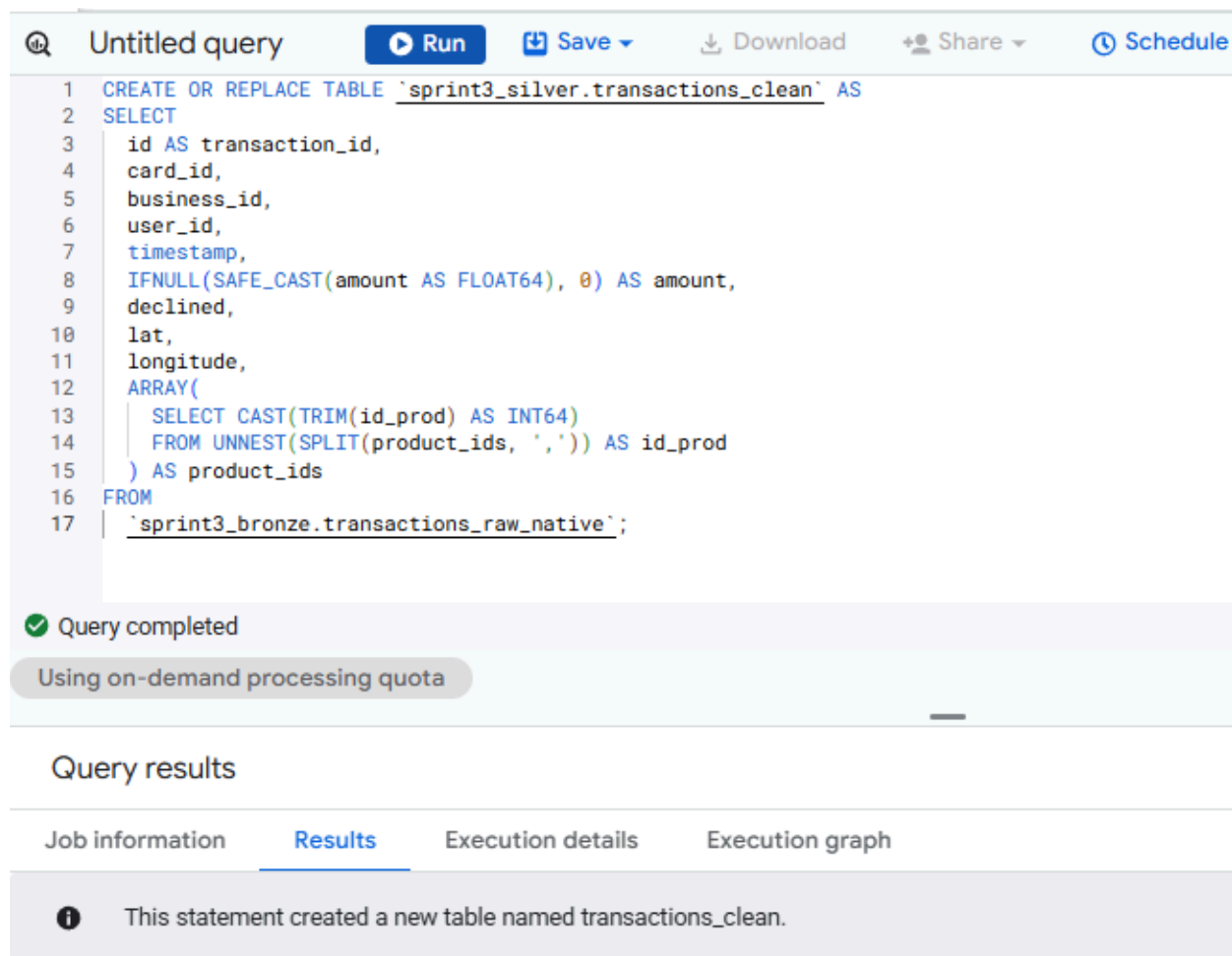
» Dates Reals: Converteix el camp timestamp (que és STRING) a tipus TIMESTAMP real.

» Coordenades: Assegura que lat i longitude siguin FLOAT64 (utilitza SAFE_CAST per seguretat).

» Desglose de Productos: Transforma la cadena de texto product_ids en un ARRAY de enteros.

- Input: "1, 2, 3" (String)
 - Output: [1, 2, 3] (Array/Lista de enteros)
 - » Resta de camps: Mantén-los igual.
- Usaremos la tabla 'transactions_raw_native' para gastar menos recursos, ya sabemos su estructura y tipos, así que no será necesario cambiar el tipo del campo 'timestamp'. Creamos la consulta siguiendo las instrucciones:

```
CREATE OR REPLACE TABLE `sprint3_silver.transactions_clean` AS
SELECT
  id AS transaction_id,
  card_id,
  business_id,
  user_id,
  timestamp,
  IFNULL(SAFE_CAST(amount AS FLOAT64), 0) AS amount,
  declined,
  lat,
  longitude,
  ARRAY(
    SELECT CAST(TRIM(id_prod) AS INT64)
    FROM UNNEST(SPLIT(product_ids, ',')) AS id_prod
  ) AS product_ids
FROM
  `sprint3_bronze.transactions_raw_native`;
```



The screenshot shows a SQL query editor interface. At the top, there's a toolbar with buttons for 'Run', 'Save', 'Download', 'Share', and 'Schedule'. The query text is as follows:

```
1 CREATE OR REPLACE TABLE `sprint3_silver.transactions_clean` AS
2 SELECT
3   id AS transaction_id,
4   card_id,
5   business_id,
6   user_id,
7   timestamp,
8   IFNULL(SAFE_CAST(amount AS FLOAT64), 0) AS amount,
9   declined,
10  lat,
11  longitude,
12  ARRAY(
13    SELECT CAST(TRIM(id_prod) AS INT64)
14    FROM UNNEST(SPLIT(product_ids, ',')) AS id_prod
15  ) AS product_ids
16 FROM
17   `sprint3_bronze.transactions_raw_native`;
```

Below the query, a green checkmark icon indicates 'Query completed'. A grey banner below that says 'Using on-demand processing quota'. The 'Query results' section is visible, with tabs for 'Job information', 'Results' (selected), 'Execution details', and 'Execution graph'. A message in the 'Results' tab states: 'This statement created a new table named transactions_clean.'

Exercici 3: Unificació d'Usuaris (UNION)

Tenim els usuaris fragmentats per regió.

✚ Tasca:

Crea la taula `sprint3_silver.users_combined`. Utilitza `UNION ALL` per unificar els usuaris dels EUA i Europa en una única llista mestra. Afegeix una columna calculada `origin` per saber d'on venen.

» Estandardització de Noms: La columna original `id` és ambigua; reanomena-la a `user_id`.

- Averiguamos el esquema de las dos tablas de usuarios:

```
SELECT
  table_name,
  column_name,
  data_type
```

```
FROM
`sprint3_bronze.INFORMATION_SCHEMA.COLUMN_FIELD_PATHS`
WHERE
table_name IN ('american_users_raw', 'european_users_raw')
ORDER BY
column_name, table_name;
```

Untitled query

Run

Save

Download

Share

Schedule

Open in

More

1

SELECT

2

table_name,

3

column_name,

4

data_type

5

FROM

6

`sprint3\_bronze.INFORMATION\_SCHEMA.COLUMN\_FIELD\_PATHS`

7

WHERE

8

table_name IN ('american_users_raw', 'european_users_raw')

9

ORDER BY

10

column_name, table_name;

This query will process 10 MB when run.

Using on-demand processing quota

Query results

Chat

Save results

Job information	Results	Visualization	JSON	Execution details	Execution graph
Row	table_name	column_name	data_type		
1	american_users_raw	address	STRING		
2	american_users_raw	birth_date	STRING		
3	american_users_raw	city	STRING		
4	american_users_raw	country	STRING		
5	american_users_raw	email	STRING		
6	american_users_raw	id	INT64		
7	american_users_raw	name	STRING		
8	american_users_raw	phone	STRING		
9	american_users_raw	postal_code	STRING		
10	american_users_raw	surname	STRING		
11	european_users_raw	address	STRING		
12	european_users_raw	birth_date	STRING		
13	european_users_raw	city	STRING		
14	european_users_raw	country	STRING		
15	european_users_raw	email	STRING		
16	european_users_raw	id	INT64		
17	european_users_raw	name	STRING		
18	european_users_raw	phone	STRING		
19	european_users_raw	postal_code	STRING		
20	european_users_raw	surname	STRING		

Results per page: 501 – 20 of 20

- Ahora que ya conocemos todos los campos y tipos de datos, podemos observar que las dos tablas tienen el mismo esquema y tipo de datos, eso nos facilita el UNION.
Creamos el código de creación de la tabla siguiendo las instrucciones:

```
CREATE OR REPLACE TABLE `sprint3_silver.users_combined` AS
SELECT
  id AS user_id,
  name,
  surname,
  email,
  phone,
  birth_date,
  address,
  city,
  postal_code,
  country,
  'USA' AS origin
FROM
  `sprint3_bronze.american_users_raw`
```

UNION ALL

```
SELECT
  id AS user_id,
  name,
  surname,
  email,
  phone,
  birth_date,
  address,
  city,
  postal_code,
  country,
  'Europe' AS origin
FROM
  `sprint3_bronze.european_users_raw`;
```

The screenshot shows a SQL query editor interface. At the top, there's a toolbar with buttons for 'Run', 'Save', 'Download', 'Share', 'Schedule', and 'Open'. The query text is as follows:

```
1 CREATE OR REPLACE TABLE `sprint3_silver.users_combined` AS
2 SELECT
3   id AS user_id,
4   name,
5   surname,
6   email,
7   phone,
8   birth_date,
9   address,
10  city,
11  postal_code,
12  country,
13  'USA' AS origin
14 FROM
15   `sprint3_bronze.american_users_raw`
16
17 UNION ALL
18
19 SELECT
20   id AS user_id,
21   name,
22   surname,
23   email,
24   phone,
25   birth_date,
26   address,
27   city,
28   postal_code,
29   country,
30   'Europe' AS origin
31 FROM
32   `sprint3_bronze.european_users_raw`;
```

Below the query, a status bar indicates 'Query completed' with a green checkmark. Underneath, there's a section for 'Query results' with tabs for 'Job information', 'Results', 'Execution details', and 'Execution graph'. The 'Results' tab is selected. A message box at the bottom states: 'This statement created a new table named users_combined.'

Exercici 4: Materialització de Companyies i Targetes de Crèdit

Escenari:

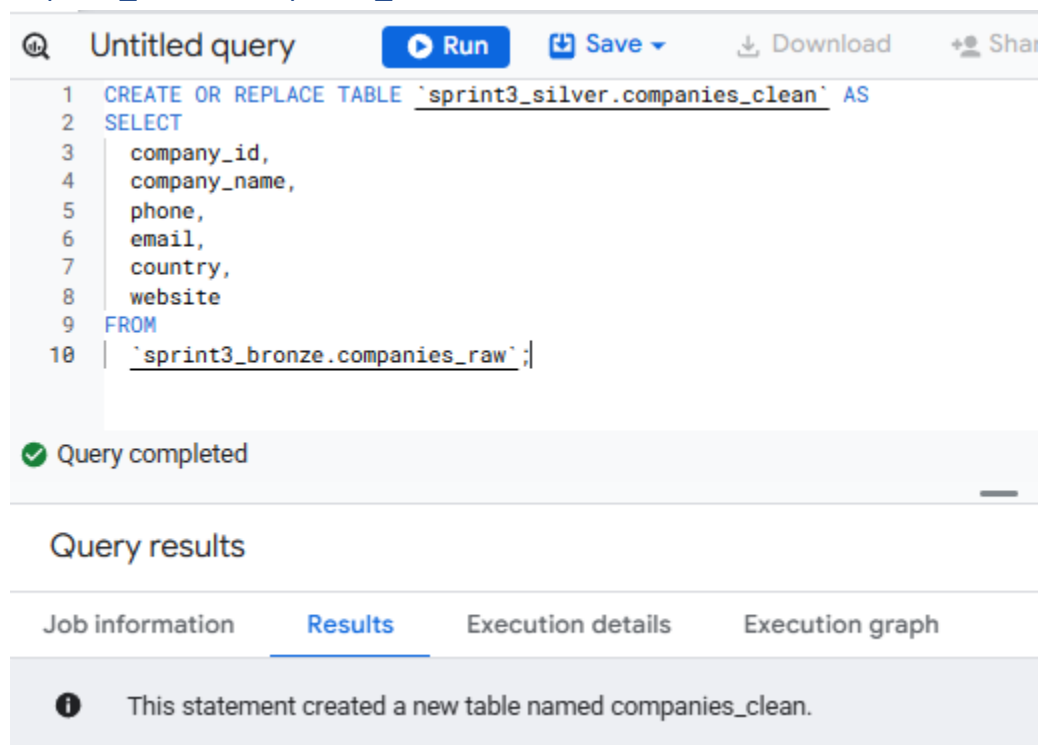
Per completar el model de dades a la capa Silver, ens falten dues dimensions clau que actualment depenen de fitxers CSV externs: Companyies i Targetes de Crèdit. No podem dependre de fitxers solts per a l'anàlisi final. Hem d'importar aquestes dades a taules natives (Silver) i aprofitar per corregir formats.

✂ Tasques:

» 1. Crea la taula `sprint3_silver.companies_clean`. Copia les dades tal qual, però assegura't que sigui una taula nativa de BigQuery (no una vista externa).

- Creamos la declaración para crear la tabla 'companies_clean' a Silver:

```
CREATE OR REPLACE TABLE `sprint3_silver.companies_clean` AS
SELECT
  company_id,
  company_name,
  phone,
  email,
  country,
  website
FROM
`sprint3_bronze.companies_raw`;
```



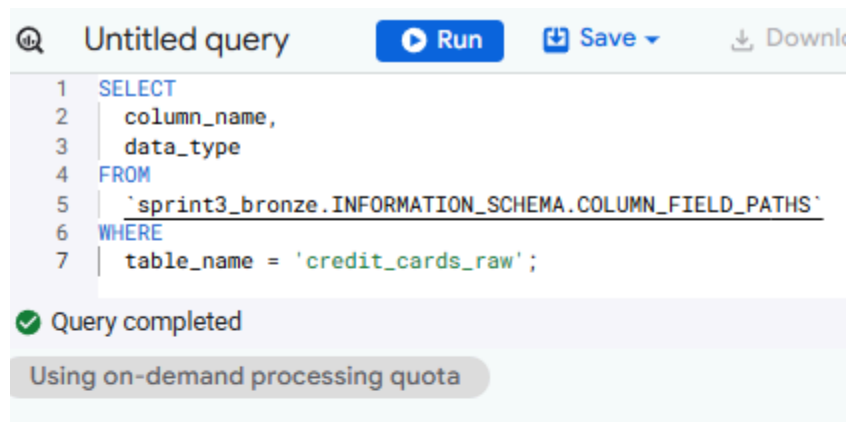
The screenshot displays the BigQuery interface. At the top, there's a toolbar with 'Run', 'Save', 'Download', and 'Share' buttons. Below it, the SQL query is entered in a text area, with line numbers 1 through 10 on the left. The query is: `CREATE OR REPLACE TABLE `sprint3_silver.companies_clean` AS SELECT company_id, company_name, phone, email, country, website FROM `sprint3_bronze.companies_raw`;`. Below the query editor, a green checkmark and the text 'Query completed' indicate success. Underneath, the 'Query results' section is visible, with tabs for 'Job information', 'Results' (which is selected), 'Execution details', and 'Execution graph'. In the 'Results' tab, a message with an information icon states: 'This statement created a new table named companies_clean.'

» 2. Crea la taula `sprint3_silver.credit_cards_clean`. Copia les dades tal qual, però assegura't que sigui una taula nativa de BigQuery (no una vista externa).

Si cal, reanomenar l'id segons correspongui.

- Averiguamos el esquema de la tabla 'credit_cards_raw':

```
SELECT
  column_name,
  data_type
FROM
  `sprint3_bronze.INFORMATION_SCHEMA.COLUMN_FIELD_PATHS`
WHERE
  table_name = 'credit_cards_raw';
```



Query results

Job information		Results	Visualization	JSON	Export
Row	column_name	data_type			
1	id	STRING			
2	user_id	INT64			
3	iban	STRING			
4	pan	STRING			
5	pin	INT64			
6	cvv	INT64			
7	track1	STRING			
8	track2	STRING			
9	expiring_date	DATE			

- Creamos la consulta para crear la tabla 'credit_cards_clean' a Silver:

```
CREATE OR REPLACE TABLE `sprint3_silver.credit_cards_clean` AS
SELECT
  id AS card_id,
  user_id,
  iban,
  pan,
```

```

pin,
cvv,
track1,
track2,
expiring_date
FROM
`sprint3_bronze.credit_cards_raw`;

```

The screenshot shows the Google Cloud BigQuery console interface. On the left, the 'Search BigQuery resources' sidebar is visible, listing various datasets like 'companies_raw', 'credit_cards_raw', 'european_users_raw', 'products_raw', 'transactions_raw', and 'transactions_raw_n'. The 'sprint3_silver' dataset is expanded, showing 'companies_clean', 'credit_cards_clean', 'products_clean', and 'transactions_clean'. The main panel displays an 'Untitled query' editor with the following SQL code:

```

1 CREATE OR REPLACE TABLE `sprint3_silver.credit_cards_clean` AS
2 SELECT
3   id AS card_id,
4   user_id,
5   iban,
6   pan,
7   pin,
8   cvv,
9   track1,
10  track2,
11  expiring_date
12 FROM
13  `sprint3_bronze.credit_cards_raw`;

```

Below the query editor, a status bar indicates 'Query completed'. The 'Query results' section is active, showing a message: 'This statement created a new table named credit_cards_clean.'

Nivell 3: Presentació de Dades i Creació de Vistes

Escenari:

Al nivell anterior vas actuar com a Data Engineer netejant les dades (_clean). Ara toca posar-se el barret d'Analista de Negoci. El problema actual és que les taules estan separades i algunes tenen formats difícils d'explotar (com la llista de productes comprimida en una sola cel·la). En aquest nivell crearàs Vistes per facilitar l'accés a la informació i generaràs informes finals.

🎯 Objectius:

- » Crear Vistes (CREATE VIEW) per encapsular lògica de negoci complexa.
- » Resoldre el repte de dades agrupades en strings (SPLIT + UNNEST).

» Exportar resultats per a tercers (Reverse ETL).

Exercici 1: La Vista de Màrqueting (Lògica de Negoci)

Màrqueting necessita segmentar els clients corporatius, però no saben fer JOINS. La teva tasca és deixar-los la informació preparada.

✚ Tasca:

Crea una vista anomenada `sprint3_gold.v_marketing_kpis` que mostri la següent informació per a cada companyia:

- » Nom de la companyia, Telèfon i País (origen: `companies_clean`).
- » Mitjana de compra (AVG(amount) de `transactions_clean`).
- » Classificació de Client (Lògica):
 - Crea una columna calculada anomenada `client_tier`.
 - Si la mitjana de compra és superior a 260€, etiqueta com a "Premium".
 - Si és igual o inferior, etiqueta com a "Standard".

- Creamos la Vista siguiendo las instrucciones de transformación:

```
CREATE OR REPLACE VIEW `sprint3_gold.v_marketing_kpis` AS
SELECT
  c.company_name,
  c.phone,
  c.country,
  ROUND(AVG(t.amount), 2) AS avg_purchase,
  IF(AVG(t.amount) > 260, 'Premium', 'Standard') AS client_tier
FROM
  `sprint3_silver.companies_clean` AS c
JOIN
  `sprint3_silver.transactions_clean` AS t
  ON c.company_id = t.business_id
GROUP BY
  c.company_name,
  c.phone,
  c.country;
```

The screenshot displays the Google Cloud BigQuery interface. On the left, a sidebar shows a project named 'sprint3-analytics-cristina-f-t' with a search bar and a list of datasets including 'companies_raw', 'credit_cards_raw', 'european_users_raw', 'products_raw', 'transactions_raw', 'transactions_raw_n', 'sprint3_gold', 'product_sales_ranki', 'v_marketing_kpis', 'sprint3_silver', 'companies_clean', 'credit_cards_clean', 'products_clean', and 'transactions_clean'. The main panel shows a query editor with the following SQL code:

```
1 CREATE OR REPLACE VIEW `sprint3_gold.v_marketing_kpis` AS
2 SELECT
3   c.company_name,
4   c.phone,
5   c.country,
6   ROUND(AVG(t.amount), 2) AS avg_purchase,
7   IF(AVG(t.amount) > 260, 'Premium', 'Standard') AS client_tier
8 FROM
9   `sprint3_silver.companies_clean` AS c
10 JOIN
11   `sprint3_silver.transactions_clean` AS t
12   ON c.company_id = t.business_id
13 GROUP BY
14   c.company_name,
15   c.phone,
16   c.country;
```

Below the query editor, a status bar indicates 'Query completed'. The 'Query results' section is visible, with tabs for 'Job information', 'Results', 'Execution details', and 'Execution graph'. A message states: 'This statement replaced the view named v_marketing_kpis.'

👉 Entrega:

Realitza una consulta `SELECT *` sobre la teva nova vista, ordenant els resultats perquè apareguin primer els clients "Premium" i, dins d'aquests, els que tinguin major mitjana de compra.

- Creamos la consulta a la vista siguiendo las instrucciones:

```
SELECT * FROM `sprint3_gold.v_marketing_kpis`
ORDER BY
  client_tier ASC,
  avg_purchase DESC;
```

Untitled query

RunSaveDownloadShareScheduleOpen inMore

```
1 SELECT * FROM `sprint3_gold.v_marketing_kpis`
2 ORDER BY
3   client_tier ASC,
4   avg_purchase DESC;
```

Query completed

Using on-demand processing quota

Query results

ChatSave results

Job information	Results	Visualization	JSON	Execution details	Execution graph
Row	company_name	phone	country	avg_purchase	client_tier
37	Quisque Libero LLC	01 45 48 71 11	China	260.51	Premium
38	Amet Lorem LLP	05 47 44 68 02	Spain	260.47	Premium
39	Ante Iaculis Nec Foundation	08 23 04 99 53	New Zealand	260.34	Premium
40	Integer Mollis Corp.	03 12 20 45 24	Italy	260.27	Premium
41	Aliquam Iaculis Lacus Corp.	04 43 07 91 26	Belgium	260.26	Premium
42	Placerat LLP	05 43 67 24 41	Netherlands	260.21	Premium
43	Tempor Diam Institute	05 01 94 80 87	Netherlands	260.1	Premium
44	Magna A Neque Industries	04 14 44 64 62	Australia	260.04	Premium
45	Risus Associates	04 25 82 54 31	Australia	259.8	Standard
46	Nunc Interdum Incorporated	05 18 15 48 13	Germany	259.32	Standard
47	Lorem Ipsum Dolor Corp.	09 03 14 76 02	Sweden	259.07	Standard
48	Etiam Bibendum Fermentum In...	07 46 69 45 02	France	258.74	Standard
49	Egestas Nunc Sed Limited	06 01 02 70 47	Italy	258.55	Standard
50	Lorem Eu Incorporated	01 83 66 62 07	Canada	258.53	Standard

Results per page: 50 1 - 50 of 100

Exercici 2: Rànquing de Productes (La Potència dels Arrays)

Escenari de Negoci:

L'equip de vendes vol optimitzar el catàleg. Necessiten un informe a la capa Gold que mostri el rendiment real de cada producte. Gràcies a la teva bona enginyeria a la capa Silver, la taula `transactions_clean` ja té els IDs de producte perfectament organitzats en una llista (Array). Ara toca explotar aquesta estructura.

🎯 Objectiu:

Crea la taula `sprint3_gold.product_sales_ranking` que contingui l'inventari complet de productes i quantes vegades s'ha venut cadascun.

📝 Requisits de l'Informe:

- » Detall del Producte: Ha d'incloure `product_id`, `name`, `price` i `color` (vénen de la taula `products_clean`).
- » Mètrica de Negoci: Una columna nova `total_sold` que compti quantes vegades apareix aquest producte a les transaccions.

» Integritat: Han d'aparèixer tots els productes, fins i tot els que tenen 0 vendes (potser cal descatalogar-los).



Pista Tècnica:

» A transactions_clean, els productes estan "encapsulats" dins d'un Array per cada transacció.
» Primer necessitaràs "aplanar" la taula de transaccions usant UNNEST(product_ids) per poder comptar els productes individualment.
» Després, fes un encreuament (LEFT JOIN) començant per la taula de productes per no perdre aquells que no s'han venut mai.

- Creamos la consulta para la creación de la nueva tabla 'product_sales_ranking' en Gold, siguiendo las instrucciones:

```
CREATE OR REPLACE TABLE `sprint3_gold.product_sales_ranking` AS
SELECT
  p.product_id,
  p.name,
  p.price,
  p.colour,
  COUNT(t_flat.single_prod_id) AS total_sold
FROM
  `sprint3_silver.products_clean` AS p
LEFT JOIN (
  SELECT
    id_p AS single_prod_id
  FROM
    `sprint3_silver.transactions_clean`,
    UNNEST(product_ids) AS id_p
) AS t_flat
ON p.product_id = t_flat.single_prod_id
GROUP BY
  p.product_id,
  p.name,
  p.price,
  p.colour
ORDER BY
  total_sold DESC;
```

Search BigQuery resources

Show starred only

- sprint3-analytics-cristina-f-t
 - Queries
 - Notebooks
 - Data canvases
 - Data preparations
 - Pipelines
 - Connections
- sprint3_bronze
 - american_users_raw
 - companies_raw
 - credit_cards_raw
 - european_users_raw
 - products_raw
 - transactions_raw
 - transactions_raw_ni
- sprint3_gold
 - product_sales_ranki
 - v_marketing_kpis
- sprint3_silver
 - companies_clean
 - products_clean
 - transactions_clean

Untitled query

```

1 CREATE OR REPLACE TABLE `sprint3_gold.product_sales_ranking` AS
2 SELECT
3   p.product_id,
4   p.name,
5   p.price,
6   p.colour,
7   COUNT(t_flat.single_prod_id) AS total_sold
8 FROM
9   `sprint3_silver.products_clean` AS p
10 LEFT JOIN (
11   SELECT
12     id_p AS single_prod_id
13 FROM
14   `sprint3_silver.transactions_clean`,
15   UNNEST(product_ids) AS id_p
16 ) AS t_flat
17 ON p.product_id = t_flat.single_prod_id
18 GROUP BY
19   p.product_id,
20   p.name,
21   p.price,
22   p.colour
23 ORDER BY
24   total_sold DESC;

```

Query completed

Using on-demand processing quota

Query results

Job information Results Execution details Execution graph

This statement created a new table named product_sales_ranking.

Exercici 3: Exportació de Resultats

El teu màner no té accés a BigQuery i vol el llistat de "Top Productes" en un Excel per a una reunió.

✖ Tasca:

Exporta les dades de la taula `product_sales_ranking` a Google Sheets o baixa l'arxiu CSV localment.

👉 Entrega:

Una captura de pantalla on es vegi l'arxiu obert (Excel/Sheets) amb les dades correctament formatades.

- Quería hacer la exportación en código SQL pero necesitaría tener un bucket y no lo tengo, tampoco se especifica en las instrucciones que deba crear uno, así que haré la exportación de los resultados de la consulta a Google Sheets:

Untitled query

Run

Save

Download

Share

Schedule

Open in

More

1

SELECT * FROM `sprint3\_gold.product\_sales\_ranking`;

Query completed

Using on-demand processing quota

Query results

Chat

Save results

Job information

Results

Visualization

JSON

Execution details

Execution graph

Row	product_id	name	price	colour	total_sold
1	52	riverlands the duel	31.52	#727272	2654
2	29	Tully maester Tary	167.2	#111111	2635
3	21	duel Direwolf	96.9	#e2e2e2	2609
4	16	the duel warden	180.91	#666666	2608
5	66	mustafar jinn	2.12	#545454	2601
6	87	sith Jade	96.26	#e2e2e2	2598
7	48	rock Renly in	21.53	#9e9e9e	2597
8	33	duel warden	127.09	#191919	2597
9	23	riverlands north	169.96	#545454	2593

Saved to Sheets as "results-20260604-140410".

Open

X

Results per page: 50 1 – 50 of 100

results-20260604-140410 ☆ 📄 ☁

File Edit View Insert Format Data Tools Extensions Help

🔍 ↶ ↷ 🖨 📋 100% ▾ | \$ % .0+ .00 123 | Defaul... ▾ | - 10 + | B

1:1000 ▾ fx product_id

	A	B	C	D	E	F
1	product_id	name	price	colour	total_sold	
2	52	riverlands the duel	31.52	#727272	2654	
3	29	Tully maester Tarly	167.2	#111111	2635	
4	21	duel Direwolf	96.9	#e2e2e2	2609	
5	16	the duel warden	180.91	#666666	2608	
6	66	mustafar jinn	2.12	#545454	2601	
7	87	sith Jade	96.26	#e2e2e2	2598	
8	48	rock Renly in	21.53	#9e9e9e	2597	
9	33	duel warden	127.09	#191919	2597	
10	23	riverlands north	169.96	#545454	2593	
11	68	Stark Karstark	167.37	#333333	2589	
12	88	Stannis warden south	167.15	#515151	2587	
13	28	chewbacca mustafar	127.44	#efefef	2584	
14	4	warden south duel	71.89	#111111	2584	
15	73	Dorne bastard	114.09	#848484	2584	
16	31	Lannister	85.02	#3f3f3f	2580	
17	34	of north	128.48	#919191	2580	
18	38	ewok sith	55.43	#666666	2578	
19	36	Lannister	96.22	#a5a5a5	2575	
20	61	Winterfell Lannister	28.01	#848484	2575	
21	54	dooku solo	13.14	#e5e5e5	2575	

+ ≡ results-20260604-140410 ▾